

GPS Sequence Diagrams

Revision: 01
Date: 19 / 06 / 2026
Author: illi Eisenberg

Table of Contents

1. Introduction	Error! Bookmark not defined.
1.1 Purpose	Error! Bookmark not defined.
1.2 Scope	Error! Bookmark not defined.
1.3 References	Error! Bookmark not defined.
2. Driver Initialization	4
2.1 GPS Initialization Sequence	Error! Bookmark not defined.
3. Protocol Detection	Error! Bookmark not defined.
3.1 PMTK Detection Sequence	Error! Bookmark not defined.
3.2 PCAS Detection Sequence	Error! Bookmark not defined.
3.3 Detection Failure Sequence	Error! Bookmark not defined.
4. Command Transmission	Error! Bookmark not defined.
4.1 Start RMC Sequence	Error! Bookmark not defined.
4.2 Stop RMC Sequence	Error! Bookmark not defined.
4.3 Start GSV Sequence	Error! Bookmark not defined.
4.4 Get Firmware Version Sequence	Error! Bookmark not defined.
4.5 Standby Sequence	Error! Bookmark not defined.
5. NMEA Reception	Error! Bookmark not defined.
5.1 DMA Reception Sequence	Error! Bookmark not defined.
5.2 Ring Buffer Update Sequence	Error! Bookmark not defined.
5.3 Sentence Ready Notification Sequence	Error! Bookmark not defined.
6. NMEA Processing	Error! Bookmark not defined.
6.1 Sentence Validation Sequence	Error! Bookmark not defined.
6.2 Checksum Verification Sequence	Error! Bookmark not defined.
6.3 Sentence Routing Sequence	Error! Bookmark not defined.
6.4 Parser Dispatch Sequence	Error! Bookmark not defined.
7. Data Update Flow	Error! Bookmark not defined.
7.1 RMC Data Update Sequence	Error! Bookmark not defined.
7.2 GSV Data Update Sequence	Error! Bookmark not defined.
7.3 GSA Data Update Sequence	Error! Bookmark not defined.
7.4 TXT Data Update Sequence	Error! Bookmark not defined.
8. FreeRTOS Interaction	Error! Bookmark not defined.
8.1 ISR to Task Notification	Error! Bookmark not defined.
8.2 Task Wake-Up Sequence	Error! Bookmark not defined.
8.3 UART Mutex Usage	Error! Bookmark not defined.

9. Error Handling	Error! Bookmark not defined.
9.1 Invalid Sentence Sequence	Error! Bookmark not defined.
9.2 Checksum Failure Sequence	Error! Bookmark not defined.
9.3 Protocol Detection Failure Sequence	Error! Bookmark not defined.
9.4 UART Timeout Sequence	Error! Bookmark not defined.
10. Future Scenarios	39
10.1 Event-Based GPS Updates	39
10.2 Multi-Consumer Data Distribution	39
11. Summary	39
Sequence Diagrams	39
5. GPS Sequence Diagrams	40

1. Introduction

1.1 Purpose

This document provides a collection of sequence diagrams describing the runtime interactions within the GPS driver. The objective of these diagrams is to illustrate the order of operations, message exchanges, and component interactions that occur during typical GPS driver activities. The diagrams complement the information presented in the Software Architecture Document (SAD) and Software Design Document (SDD) by providing a visual representation of system behavior.

1.2 Scope

This document covers the major runtime scenarios of the GPS driver, including:

- Driver initialization
- Protocol detection
- Command transmission
- NMEA sentence reception
- NMEA sentence processing
- Parser execution
- FreeRTOS interaction
- Error handling scenarios

The document focuses on interaction sequences between software components and external GPS receivers.

Detailed implementation information, algorithms, and data structure definitions are intentionally excluded and are documented in the Software Design Document (SDD).

1.3 References

The following documents provide additional information related to the GPS driver:

Document	Description
GPS Software Architecture Document (SAD)	High-level architecture and component relationships
GPS Software Design Document (SDD)	Detailed software design and implementation
NMEA 0183 Specification	Standard GPS sentence definitions
Vendor Protocol Documentation	PMTK and PCAS command specifications
FreeRTOS Documentation	RTOS services and synchronization mechanisms

These documents should be consulted together to obtain a complete understanding of the GPS driver architecture, design, and runtime behavior.

2. GPS Initialization Sequence

2.1 Purpose

This sequence describes the complete initialization flow of the GPS driver, beginning with the application request and ending with a fully initialized GPS interface.

The sequence includes:

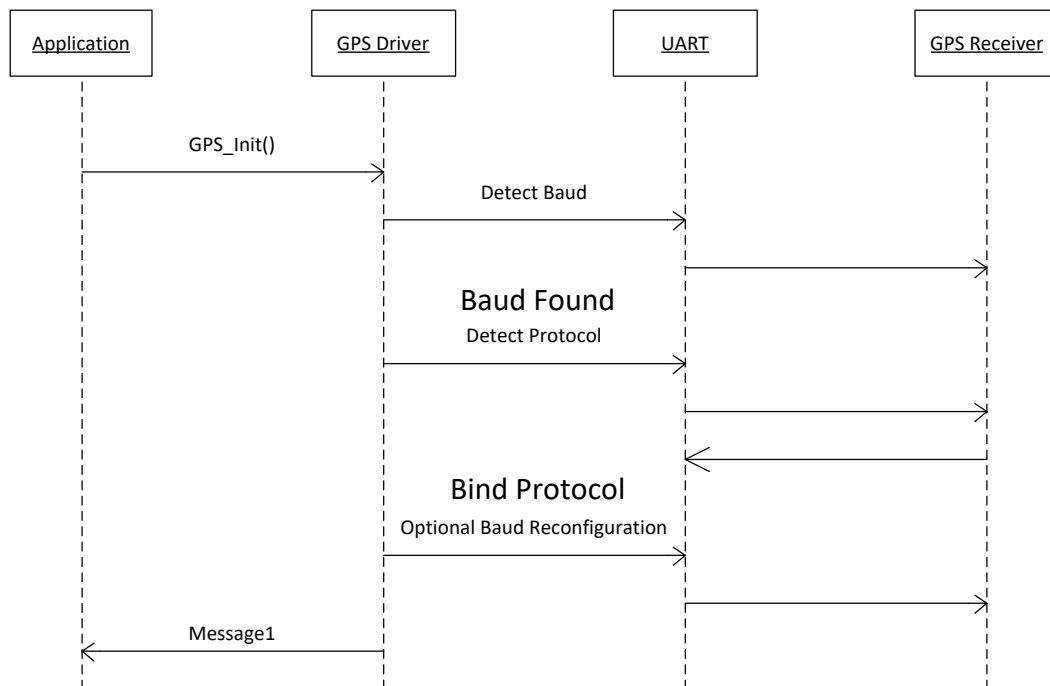
- Baud-rate detection
- Protocol detection
- Protocol binding
- Optional baud-rate reconfiguration
- Driver activation

The objective is to establish communication with the GPS receiver and prepare the driver for normal operation.

2.2 Participants

Component	Responsibility
Application	Requests GPS initialization
GPS Driver	Controls initialization process
UART Transport	Provides communication channel
GPS Receiver	Responds to protocol commands

2.3 Sequence Diagram



2.4 Sequence Description

1. The application calls `GPS_Init()`.
2. The driver performs automatic baud-rate detection by communicating with the GPS receiver through the UART transport layer.
3. After a valid response is received, the detected baud-rate is stored.
4. The driver performs protocol detection by transmitting protocol-specific commands and validating the received responses.
5. Upon successful detection, the appropriate protocol implementation is bound to the protocol interface.
6. If the detected baud-rate differs from the driver's preferred operating baud-rate, the driver reconfigures both the GPS receiver and the local UART interface.
7. The driver marks itself as initialized and returns success to the application.
8. The GPS interface is now ready for command transmission and NMEA processing.

3. Protocol Detection

3.1 PMTK Detection Sequence

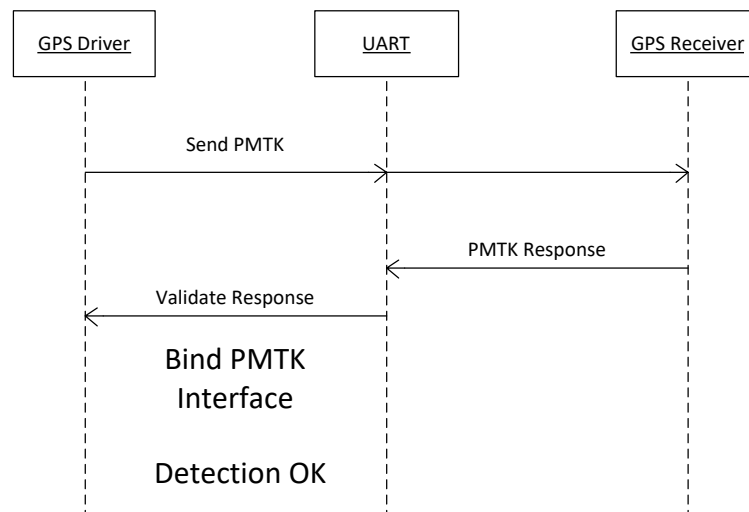
Purpose

This sequence describes successful detection of a PMTK-compatible GPS receiver.

The driver transmits a PMTK-specific command and validates the received response. Upon successful validation, the PMTK protocol interface is bound to the driver context.

Sequence Diagram

This sequence describes successful detection of a PMTK-compatible GPS receiver.



Sequence Description

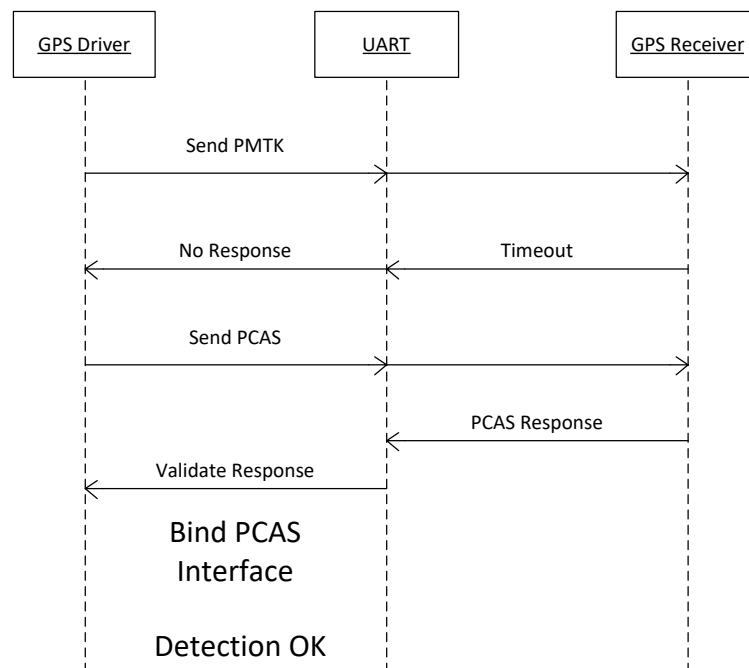
1. The driver transmits a PMTK protocol command.
2. The GPS receiver returns a valid PMTK response.
3. The driver validates the received message.
4. The PMTK protocol interface is bound to the driver context.
5. Protocol detection completes successfully.

3.2 PCAS Detection Sequence

Purpose

This sequence describes successful detection of a PCAS-compatible GPS receiver after PMTK detection has failed.

Sequence Diagram



Sequence Description

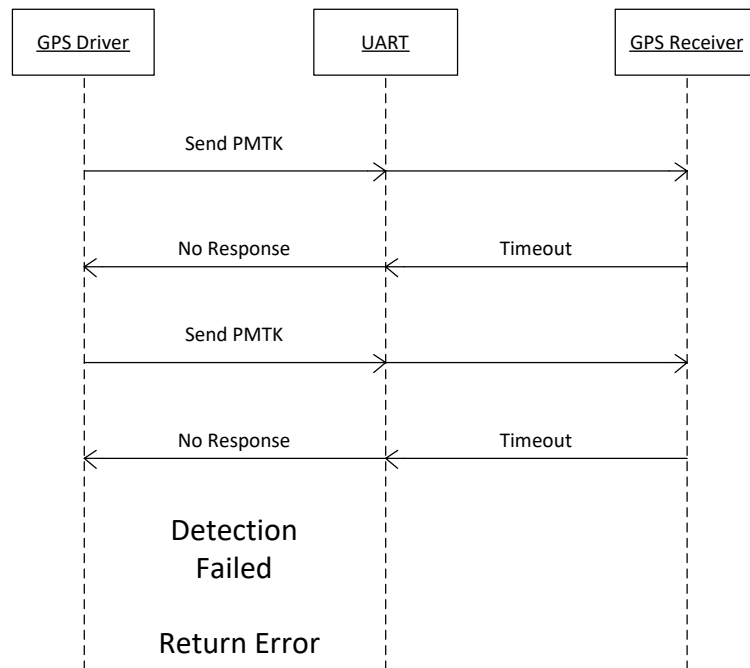
1. The driver first attempts PMTK detection.
2. No valid PMTK response is received.
3. The PMTK attempt times out.
4. The driver transmits a PCAS protocol command.
5. A valid PCAS response is received.
6. The response is validated.
7. The PCAS protocol interface is bound to the driver context.
8. Protocol detection completes successfully.

3.3 Detection Failure Sequence

Purpose

This sequence describes the behavior when none of the supported protocols can be detected.

Sequence Diagram

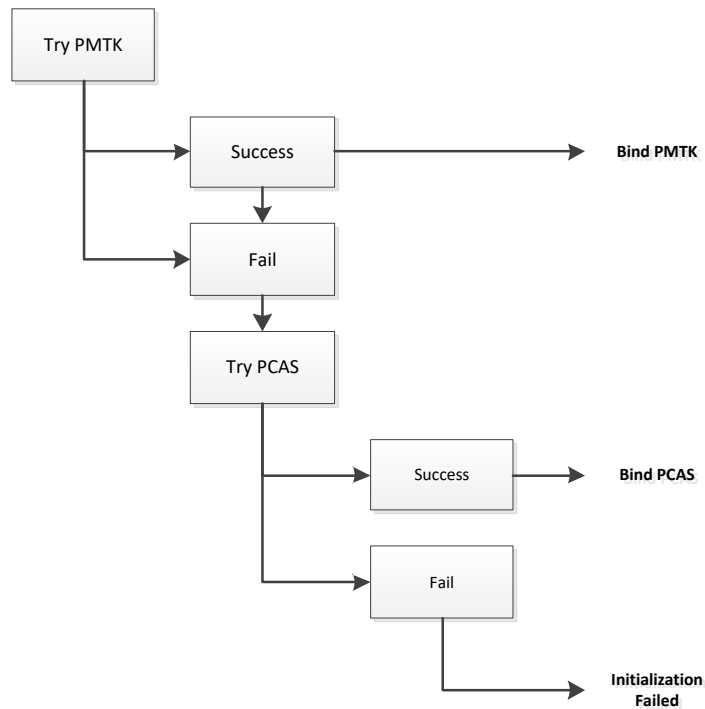


Sequence Description

1. The driver attempts PMTK detection.
2. No valid PMTK response is received.
3. The PMTK attempt times out.
4. The driver attempts PCAS detection.
5. No valid PCAS response is received.
6. The PCAS attempt times out.
7. The driver concludes that the protocol cannot be identified.
8. Initialization fails and an error is returned to the caller.

Documentation Note

The protocol detection logic follows the sequence below:

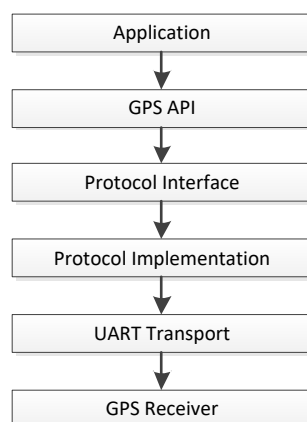


This sequence ensures automatic protocol selection without requiring user configuration

4. Command Transmission

This section describes the runtime sequences used to transmit commands from the application to the GPS receiver through the protocol abstraction layer.

All command transmissions follow the same high-level flow:



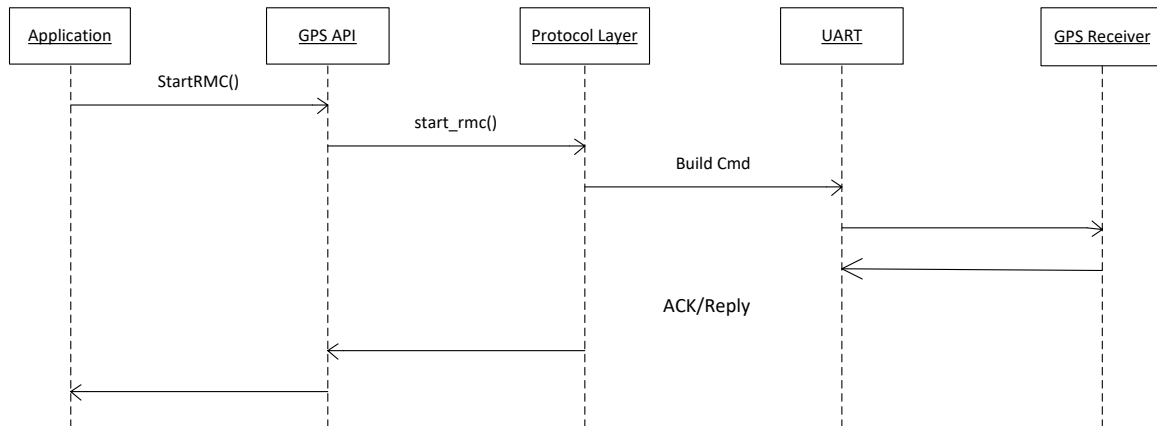
The protocol abstraction layer ensures that the application remains independent of the underlying GPS protocol.

4.1 Start RMC Sequence

Purpose

This sequence enables periodic transmission of RMC sentences from the GPS receiver.

Sequence Diagram



Sequence Description

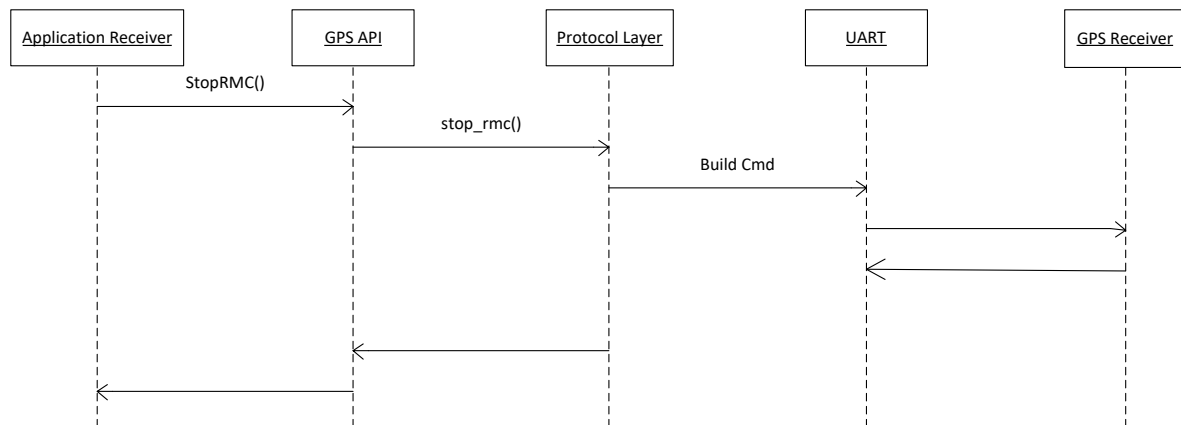
1. The application requests RMC output.
2. The GPS API invokes the bound protocol handler.
3. The protocol layer generates the appropriate command string.
4. The command is transmitted through UART.
5. The GPS receiver acknowledges the command.
6. Periodic RMC output becomes active.

4.2 Stop RMC Sequence

Purpose

This sequence disables periodic RMC sentence transmission.

Sequence Diagram



Sequence Description

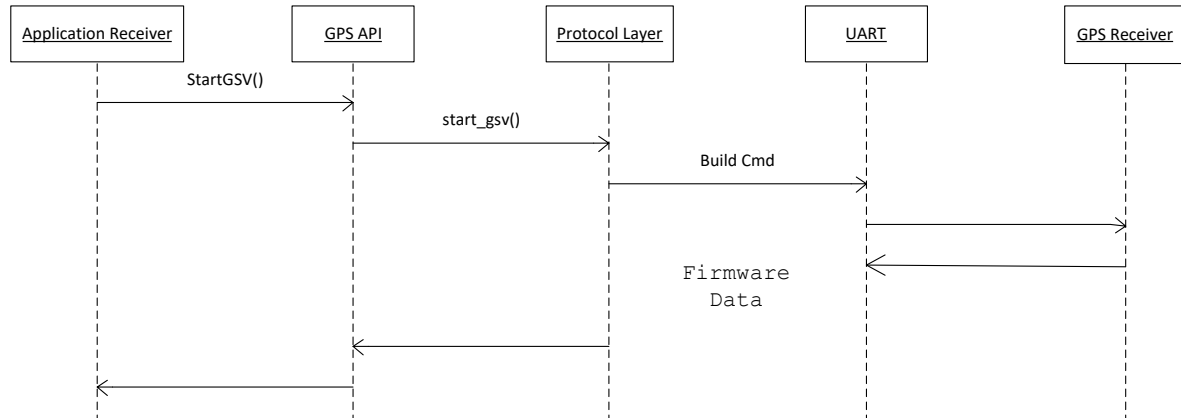
1. The application requests RMC deactivation.
2. The protocol-specific command is generated.
3. The command is transmitted to the receiver.
4. The receiver disables RMC output.
5. Command execution completes.

4.3 Start GSV Sequence

Purpose

This sequence enables transmission of satellite visibility information.

Sequence Diagram



Sequence Description

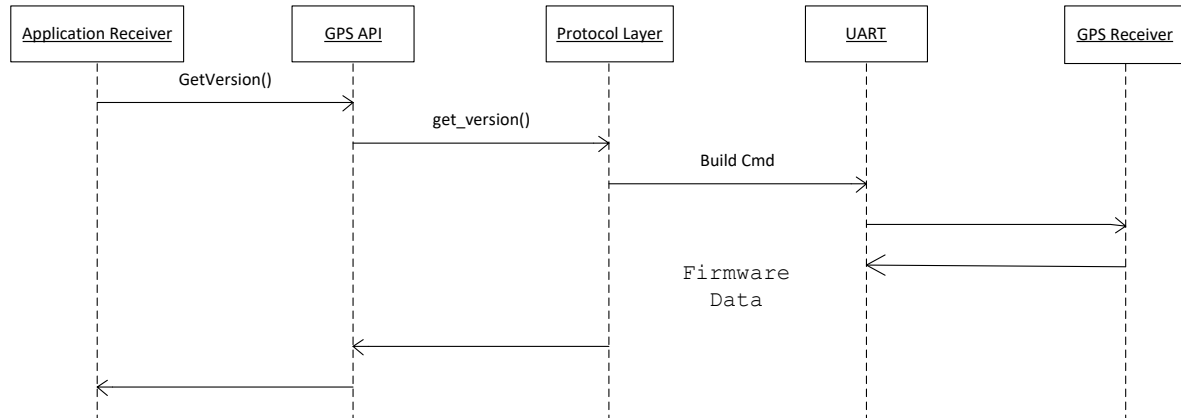
1. The application requests GSV output.
2. The protocol layer builds the corresponding command.
3. The command is transmitted through UART.
4. The receiver enables GSV reporting.
5. Satellite information becomes available through NMEA output.

4.4 Get Firmware Version Sequence

Purpose

This sequence retrieves firmware identification information from the GPS receiver.

Sequence Diagram



Sequence Description

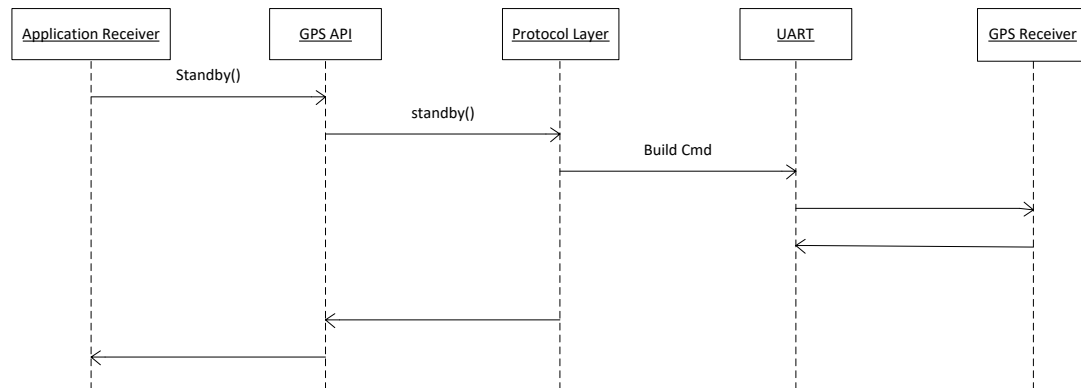
1. The application requests firmware information.
2. The protocol layer generates a version query command.
3. The command is transmitted to the receiver.
4. The receiver returns firmware information.
5. The information is parsed and stored in the firmware data structure.
6. The API returns successfully.

4.5 Standby Sequence

Purpose

This sequence places the GPS receiver into a low-power standby state.

Sequence Diagram

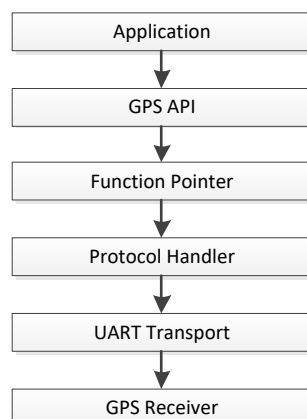


Sequence Description

1. The application requests standby mode.
2. The protocol layer generates the standby command.
3. The command is transmitted through UART.
4. The GPS receiver enters low-power mode.
5. Normal NMEA output is suspended until the receiver is reactivated.

Design Note

Although the commands differ in functionality, all command APIs use the same protocol abstraction mechanism:



This design allows the application to remain completely independent of PMTK, PCAS, or any future protocol implementation.

5. NMEA Reception

This section describes the flow of incoming NMEA data from the GPS receiver through the UART DMA mechanism into the ring buffer system.

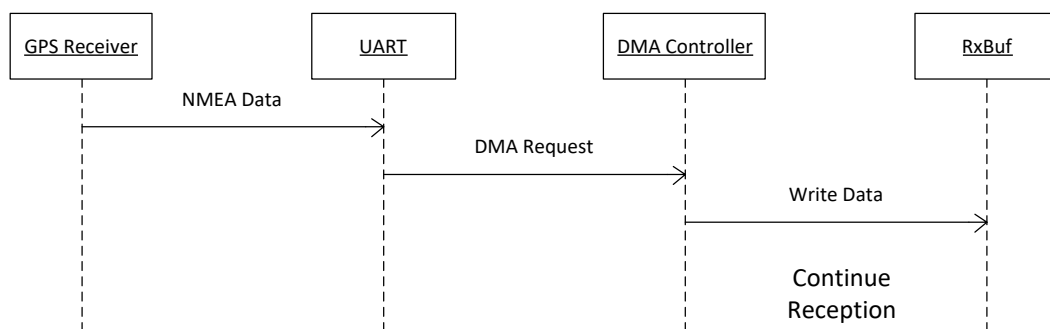
The design minimizes CPU involvement by using DMA-based reception and deferred processing.

5.1 DMA Reception Sequence

Purpose

This sequence describes how NMEA characters are received from the GPS receiver using UART DMA.

Sequence Diagram



Sequence Description

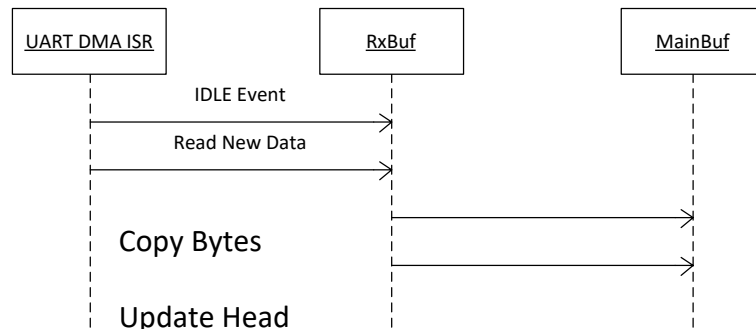
1. The GPS receiver continuously transmits NMEA sentences.
2. UART receives incoming characters.
3. DMA transfers received bytes directly into the DMA reception buffer (RxBuf).
4. No CPU intervention is required for individual characters.
5. Reception continues until an IDLE event is detected.

5.2 Ring Buffer Update Sequence

Purpose

This sequence describes how newly received DMA data is copied into the main ring buffer.

Sequence Diagram



Sequence Description

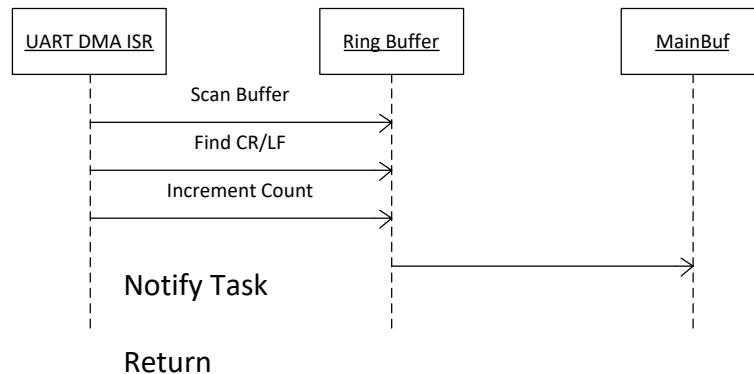
1. UART detects an IDLE condition.
2. The DMA reception callback is executed.
3. Newly received bytes are identified.
4. The bytes are copied from `RxBuf` into the main ring buffer (`MainBuf`).
5. The ring buffer head index is updated.
6. Reception immediately resumes.

5.3 Sentence Ready Notification Sequence

Purpose

This sequence describes how completed NMEA sentences are detected and how the processing task is notified.

Sequence Diagram

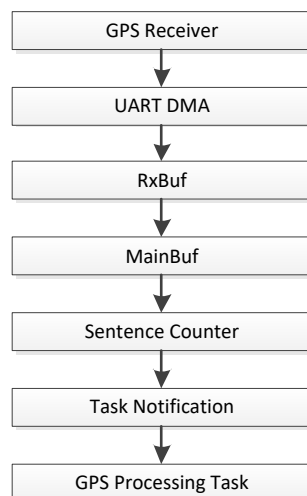


Sequence Description

1. The DMA callback scans newly received data.
2. The callback searches for sentence terminators (`\r\n`).
3. For each complete sentence found, the sentence counter is incremented.
4. A task notification is sent to the GPS processing task.
5. The ISR exits immediately.
6. Actual sentence extraction and parsing are performed later in task context.

Design Note

The driver intentionally separates reception from processing:



This architecture keeps interrupt execution time short while allowing sentence parsing to occur in normal task context, improving system responsiveness and reducing ISR workload.

6. NMEA Processing

This section describes the processing flow executed after a complete NMEA sentence has been extracted from the ring buffer.

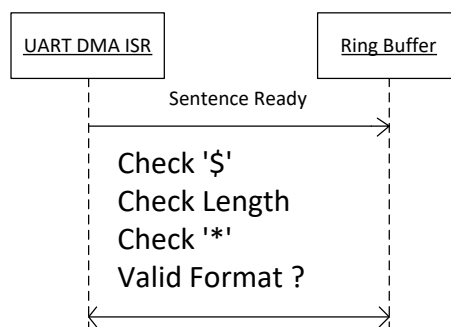
The processing pipeline performs validation, checksum verification, sentence identification, and parser dispatching before updating the internal GPS data structures.

6.1 Sentence Validation Sequence

Purpose

This sequence verifies that the received message follows the expected NMEA sentence format before further processing is performed.

Sequence Diagram



Sequence Description

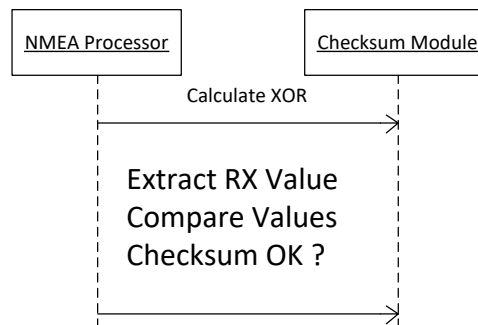
1. A complete sentence is extracted from the ring buffer.
2. The processor verifies that the sentence starts with \$.
3. The processor verifies that the sentence length is within acceptable limits.
4. The processor checks for the presence of the checksum separator (*).
5. If any validation step fails, the sentence is discarded.
6. Valid sentences proceed to checksum verification.

6.2 Checksum Verification Sequence

Purpose

This sequence verifies the integrity of the received NMEA sentence.

Sequence Diagram



Sequence Description

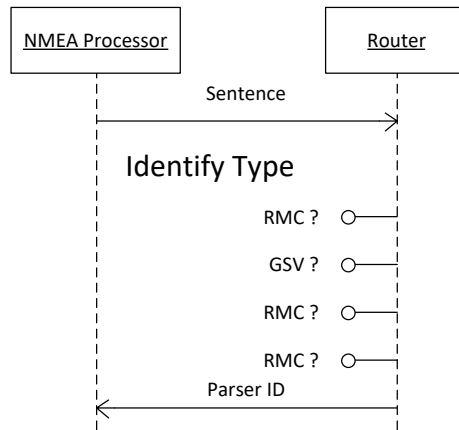
1. The processor calculates the checksum using all characters between \$ and *.
2. The received checksum value is extracted from the sentence.
3. The calculated checksum is compared with the received checksum.
4. If the values match, processing continues.
5. If verification fails, the sentence is discarded.

6.3 Sentence Routing Sequence

Purpose

This sequence identifies the sentence type and selects the appropriate parser.

Sequence Diagram



Sequence Description

1. A validated sentence is passed to the routing module.
2. The sentence identifier is extracted.
3. The router determines the sentence type.
4. The matching parser is selected.
5. The selected parser is returned to the processing engine.

Examples:

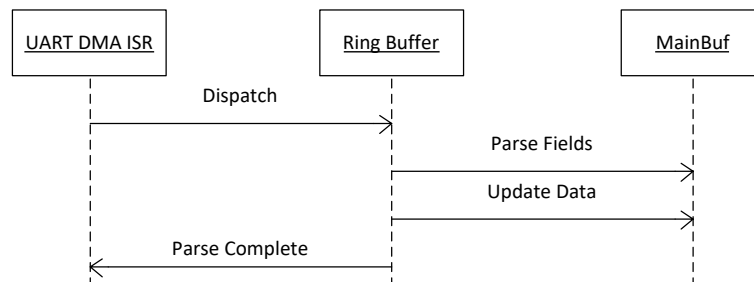
Sentence	Parser
GPRMC	RMC Parser
GNRMC	RMC Parser
GPGSV	GSV Parser
GNGSV	GSV Parser
GPGSA	GSA Parser
GPTXT	TXT Parser

6.4 Parser Dispatch Sequence

Purpose

This sequence executes the selected parser and updates the corresponding data structure.

Sequence Diagram

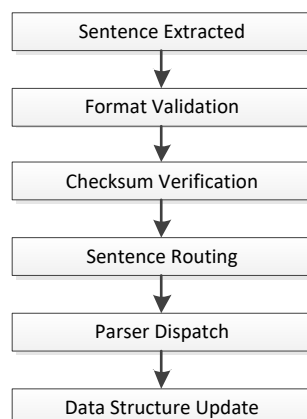


Sequence Description

1. The routing module selects the appropriate parser.
2. The parser receives the raw NMEA sentence.
3. Individual fields are extracted and converted.
4. Parsed values are stored in the relevant GPS data structure.
5. The parser returns a success or failure status.
6. Processing of the current sentence is completed.

Design Note

The NMEA processing pipeline follows a strict sequence:



This staged approach prevents invalid or corrupted data from reaching the application layer and keeps individual processing responsibilities clearly separated.

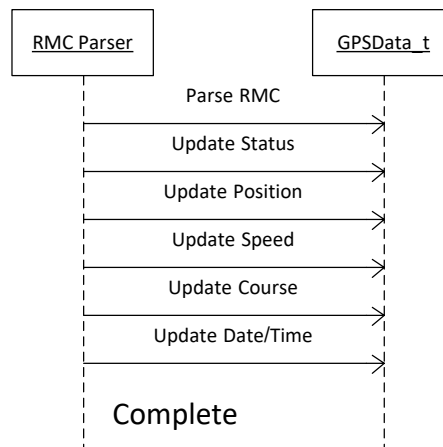
7. Data Update Flow

This section describes how successfully parsed NMEA data is stored in the internal driver data structures. Each parser updates a dedicated structure that represents a specific category of GPS information. The update operation is performed only after sentence validation, checksum verification, and successful parsing have completed.

Purpose

This sequence describes how navigation data extracted from an RMC sentence is stored in the GPS data structure.

Sequence Diagram



Sequence Description

1. The RMC parser receives a validated RMC sentence.
2. Navigation status is updated.
3. Latitude and longitude are updated.
4. Speed information is updated.
5. Course information is updated.
6. UTC date and time fields are updated.
7. The GPS data structure now contains the latest navigation information.

Updated Fields

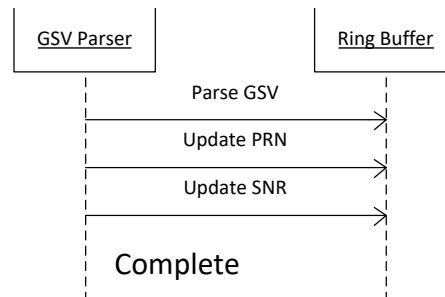
```
GPSData_t
├── status
├── latitude
├── longitude
├── speed
├── course
├── Date
└── Time
```

7.2 GSV Data Update Sequence

Purpose

This sequence describes how satellite visibility information is stored.

Sequence Diagram



Sequence Description

1. The GSV parser processes one GSV sentence.
2. Satellite information is extracted.
3. Satellite identifiers are updated.
4. Signal-to-noise ratio values are updated.
5. The satellite database is refreshed.

Updated Fields

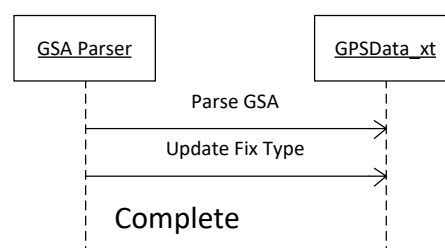
```
GPSsats_t
├── Satellite ID
└── SNR
```

7.3 GSA Data Update Sequence

Purpose

This sequence describes how fix quality information is updated.

Sequence Diagram



Sequence Description

1. The GSA parser receives a validated GSA sentence.
2. The current fix type is extracted.
3. Satellite solution quality information is processed.
4. DOP values are updated if supported.
5. Fix information is stored for later use.

Updated Fields

```
GPSTData_xt  
└─ fix
```

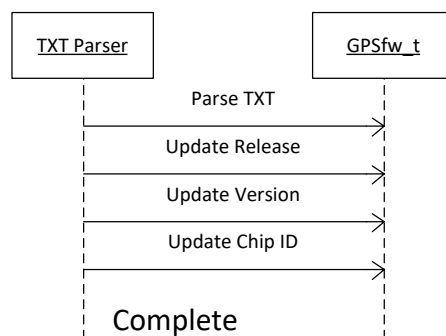
The exact fields depend on the implementation and enabled parser features.

7.4 TXT Data Update Sequence

Purpose

This sequence describes how firmware and device identification information is stored.

Sequence Diagram



Sequence Description

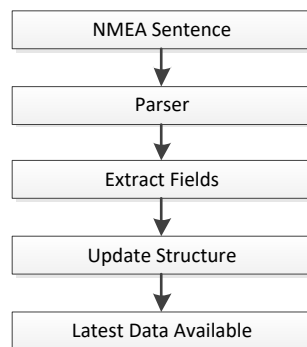
1. The TXT parser processes a firmware information sentence.
2. Firmware release information is extracted.
3. Build or version information is updated.
4. Chip identification information is updated.
5. Manufacturer information is updated when available.
6. The firmware information structure is refreshed.

Updated Fields

```
GPSfw_xt
├── ReleaseStr
├── Build_ID
├── Internal_USE_1
├── Internal_USE_2
└── Manufacturer
```

Data Update Summary

The data update process can be summarized as follows:



Each parser is responsible only for updating the structure associated with its sentence type. This separation simplifies maintenance and prevents unrelated data from being modified during processing.

8. FreeRTOS Interaction

This section describes the interaction between the GPS driver and the FreeRTOS kernel.

The driver minimizes ISR execution time by performing only the minimum required work inside interrupt context and deferring processing to task context through task notifications.

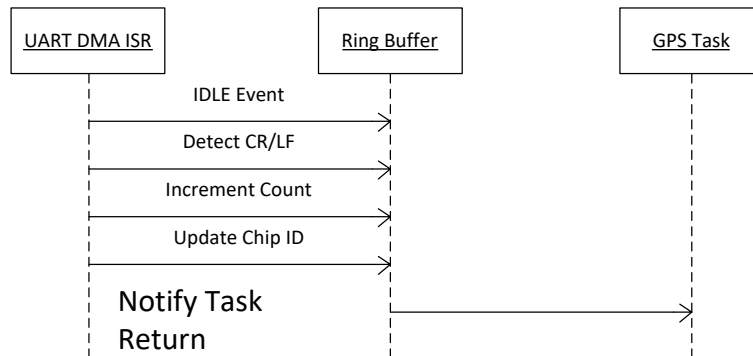
Synchronization mechanisms are used to protect shared UART resources when commands are transmitted.

8.1 ISR to Task Notification

Purpose

This sequence describes how the DMA callback notifies the GPS processing task that one or more complete NMEA sentences are available.

Sequence Diagram



Sequence Description

1. UART DMA generates an IDLE event.
2. The DMA callback updates the ring buffer.
3. The callback detects one or more complete NMEA sentences.
4. The sentence counter is updated.
5. A task notification is sent to the GPS processing task.
6. The ISR exits immediately.
7. All parsing activities remain in task context.

Design Objective

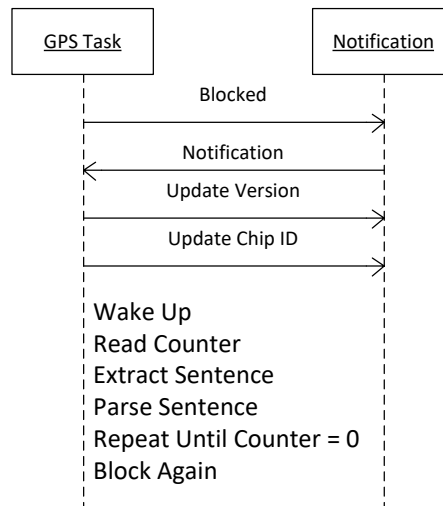
Design objective: keeping the ISR execution short and deterministic.

8.2 Task Wake-Up Sequence

Purpose

This sequence describes how the GPS task wakes up and processes pending NMEA sentences.

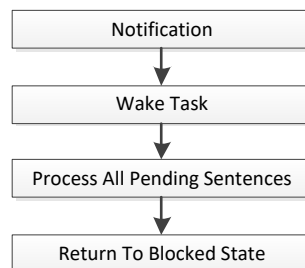
Sequence Diagram



Sequence Description

1. The GPS task waits for a notification.
2. The ISR sends a notification.
3. The task becomes ready and resumes execution.
4. The task reads the number of pending sentences.
5. Sentences are extracted from the ring buffer.
6. Each sentence is validated and parsed.
7. Processing continues until all pending sentences have been handled.
8. The task returns to the blocked state.

Processing Model



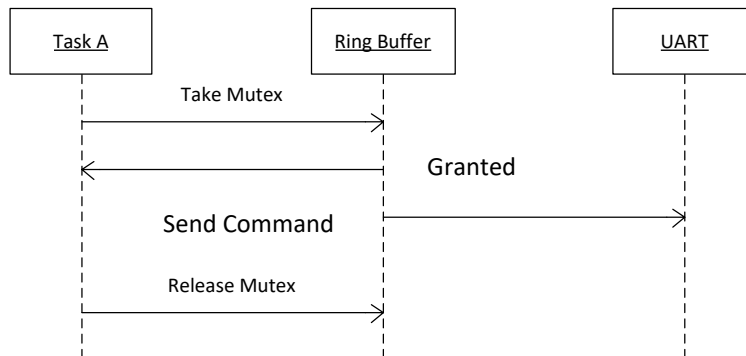
This approach avoids excessive context switching when multiple sentences arrive close together.

8.3 UART Mutex Usage

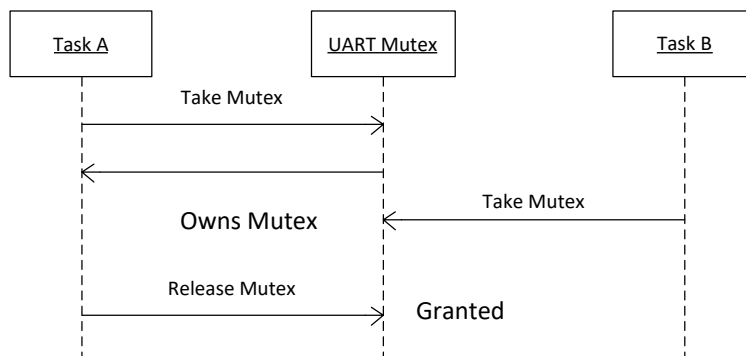
Purpose

This sequence describes how UART access is protected when multiple tasks may transmit commands to the GPS receiver.

Sequence Diagram



Competing Access Example



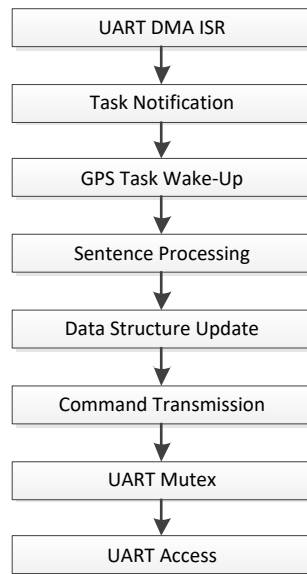
Sequence Description

1. A task requests access to the UART.
2. The task acquires the UART mutex.
3. Command transmission is performed.
4. The mutex is released after transmission completes.
5. Waiting tasks may then obtain access to the UART.

Design objective: Ensure exclusive UART access and prevent command corruption.

Only task-context code acquires the mutex. ISR code never attempts to lock or unlock synchronization objects.

FreeRTOS Interaction Summary



This design keeps interrupt latency low, provides deterministic task scheduling, and ensures safe access to shared communication resources.

9. Error Handling

This section describes the sequences executed when abnormal conditions are detected during GPS driver operation.

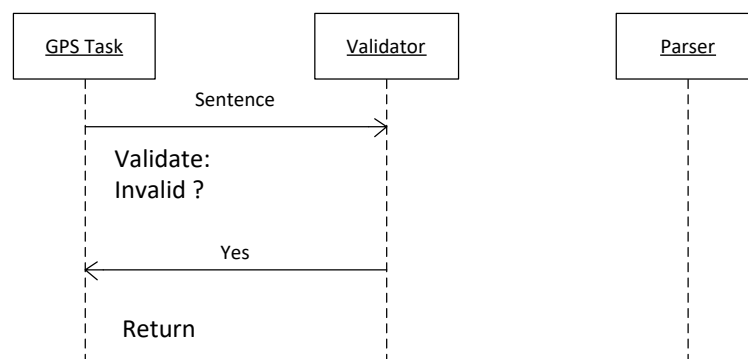
The driver is designed to detect, isolate, and recover from errors without affecting normal system operation whenever possible.

9.1 Invalid Sentence Sequence

Purpose

This sequence describes how malformed or incomplete NMEA sentences are handled.

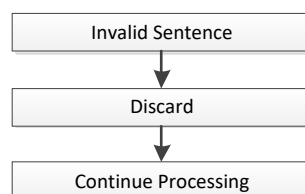
Sequence Diagram



Sequence Description

1. A sentence is extracted from the ring buffer.
2. The validator checks the sentence format.
3. Required NMEA fields are verified.
4. If the sentence is malformed or incomplete, validation fails.
5. The sentence is discarded.
6. No parser is executed.
7. Processing continues with the next available sentence.

Result

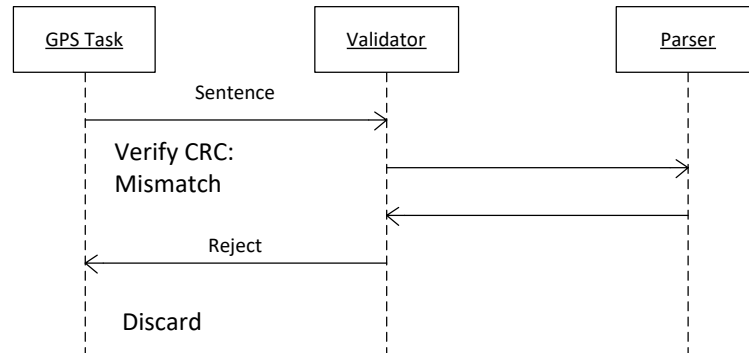


9.2 Checksum Failure Sequence

Purpose

This sequence describes how corrupted NMEA sentences are handled.

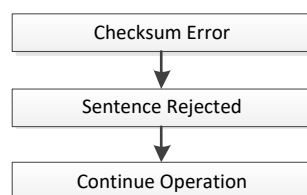
Sequence Diagram



Sequence Description

1. The sentence passes format validation.
2. The checksum calculation is performed.
3. The calculated checksum is compared with the received value.
4. If a mismatch is detected, the sentence is rejected.
5. No parser is executed.
6. Processing continues normally.

Result

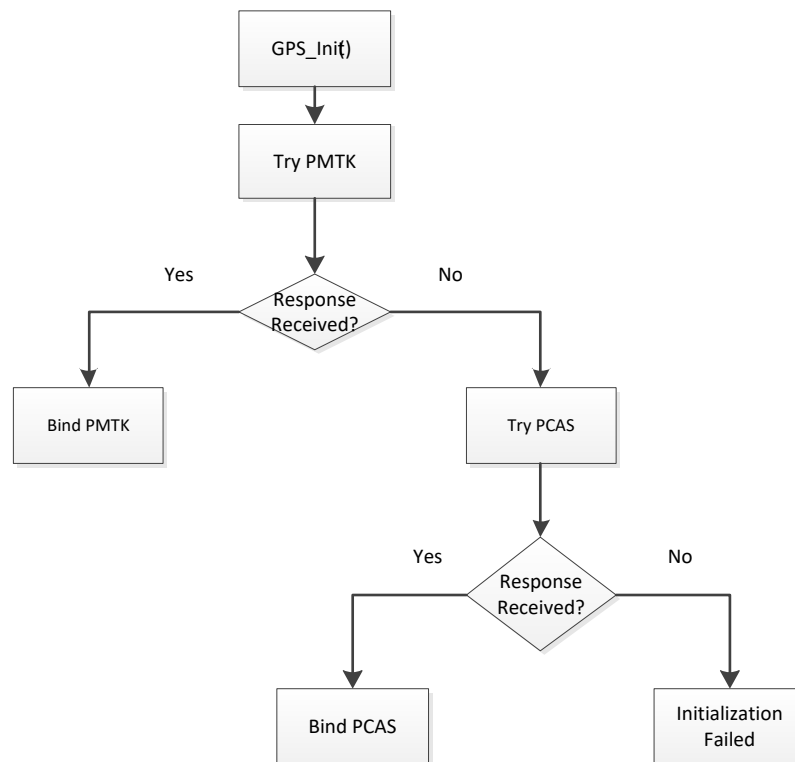


9.3 Protocol Detection Failure Sequence

Purpose

This sequence describes the initialization failure path when no supported GPS protocol is detected.

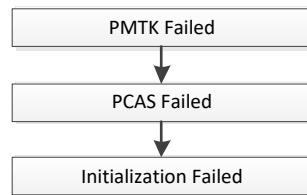
Sequence Diagram



Sequence Description

1. Initialization begins.
2. The driver attempts PMTK protocol detection.
3. If PMTK is not detected, PCAS detection is attempted.
4. If neither protocol responds, initialization fails.
5. No protocol binding occurs.
6. The driver reports initialization failure to the caller.

Result



Recovery

Recovery is application dependent and may include:

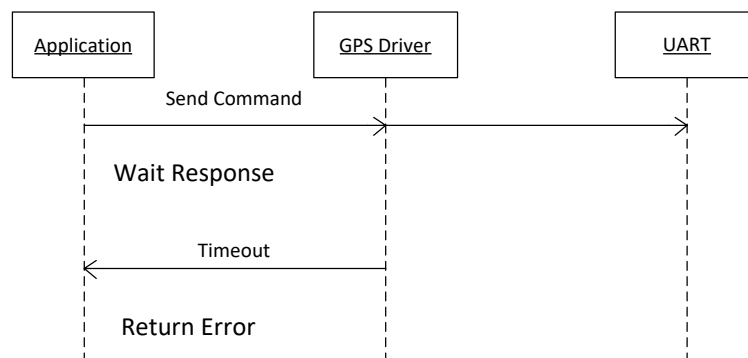
- Retrying initialization
- Selecting a different baud rate
- Reporting an error to the user
- Entering a degraded operating mode

9.4 UART Timeout Sequence

Purpose

This sequence describes how command-response timeouts are handled.

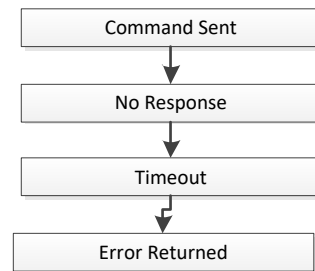
Sequence Diagram



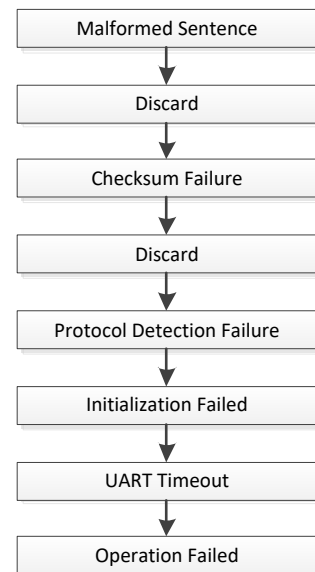
Sequence Description

1. A command is transmitted to the GPS receiver.
2. The driver waits for a response.
3. No valid response is received before the timeout expires.
4. The operation is terminated.
5. An error status is returned.
6. The application decides whether to retry or continue.

Result



Error Handling Summary



The driver follows a fail-safe approach where invalid or corrupted data is rejected as early as possible, preventing erroneous information from reaching the internal GPS data structures.

10. Future Scenarios

This section presents potential future interaction flows that may be implemented in future versions of the GPS driver. The scenarios described below are not part of the current implementation but represent planned architectural extensions.

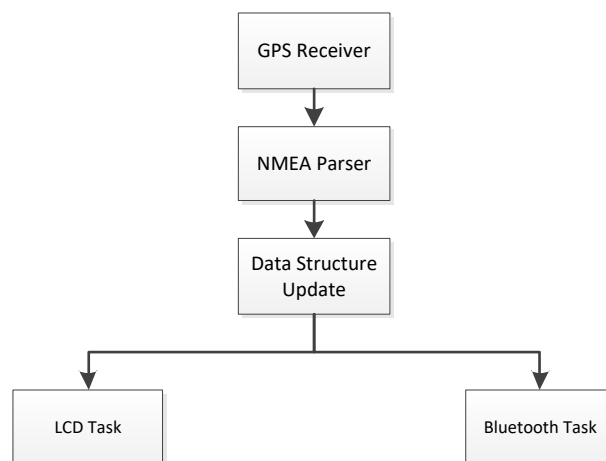
10.1 Event-Based GPS Updates

Purpose

This scenario describes a future event-driven architecture in which GPS data updates are automatically distributed whenever new information becomes available.

Instead of requiring periodic polling, consumers can subscribe to GPS update events and receive notifications when relevant data changes.

Sequence Diagram



Sequence Description

1. A new NMEA sentence is received and parsed.
2. The corresponding GPS data structure is updated.
3. The driver generates a data-update event.
4. Interested consumers receive notification of the update.
5. Each consumer processes the information independently.

Benefits

- Reduced polling activity.
- Faster propagation of GPS updates.
- Improved scalability.
- Lower CPU utilization in consumer tasks.

Example Use Cases

- Position updates.
- Satellite status changes.
- GPS fix state changes.
- Firmware information availability.

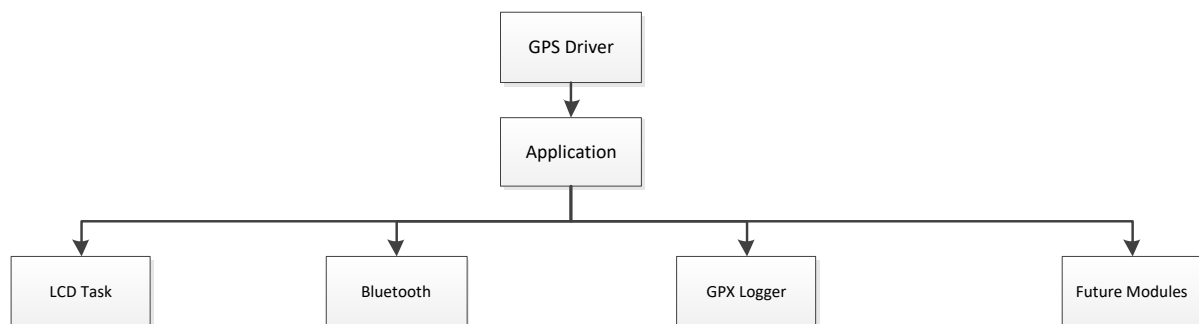
10.2 Multi-Consumer Data Distribution

Purpose

This scenario describes a future architecture where multiple software modules consume GPS information simultaneously through a centralized application layer.

This aligns with the overall system architecture in which sensor drivers remain isolated from application consumers.

Sequence Diagram



Sequence Description

1. The GPS driver receives and parses GPS information.
2. Updated data is forwarded to the application layer.
3. The application acts as the distribution point.
4. Multiple consumers receive the same data.
5. Each consumer performs its own processing without direct interaction with the GPS driver.

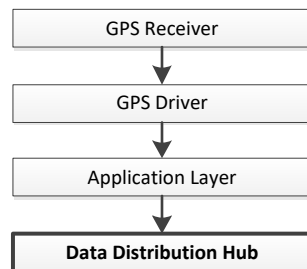
Benefits

- Clear separation of responsibilities.
- Simplified maintenance.
- Easier addition of new consumers.
- Consistent data distribution model.
- Reduced coupling between software components.

Example Consumers

- LCD user interface.
- Bluetooth communication module.
- GPX recording service.
- Remote telemetry services.
- Future cloud connectivity modules.

Future Architecture Vision



This approach allows future system growth while preserving the existing driver architecture and maintaining a single source of GPS information within the application layer.

10. Future Scenarios

10.1 Event-Based GPS Updates

10.2 Multi-Consumer Data Distribution

11. Summary

Sequence Diagrams

מטרתו:

- Initialization sequence
- Protocol detection sequence

- RMC request sequence
- GSV request sequence
- Version request sequence
- GPX logging sequence

5. GPS Sequence Diagrams

מתאר:

- Initialization
- Protocol detection
- RMC request
- GSV request
- Version request
- GPX logging flow